Experiment No. 7

Student Name: Shubham Agarwal **UID: 25MCI10091**
Branch: MCA (AI & ML) **Section/Group: 25MAM-1/A**
Semester: 2nd **Date of Performance:31/03/2026**
Subject Name: Technical Training Lab **Subject Code: 25CAH-653**

Aim of the Session:

To implement and understand different types of JOIN operations in PostgreSQL for combining data from multiple tables.

Software Requirements:

- PostgreSQL Database Server
- pgAdmin 4
- Windows Operating System

Objective of the Session:

- To apply different types of JOIN operations on four related tables—Students, Courses, Enrollments, and Departments.
- To understand the relationships between these tables using primary and foreign keys.
- To combine data from multiple tables to retrieve meaningful and structured information.
- To enhance query writing skills for real-world database analysis and reporting.

Practical Experiment Steps:

Table Creation & Record Insertion:

```
CREATE TABLE DEPARTMENTS (
   dept_id VARCHAR(10) PRIMARY KEY, dept_name VARCHAR(50)
);

CREATE TABLE STUDENTS (
   student_id VARCHAR(15) PRIMARY KEY, student_name VARCHAR(50), dept_id VARCHAR(10),
   FOREIGN KEY (dept_id) REFERENCES Departments(dept_id)
);

CREATE TABLE Courses (
   course_id VARCHAR(15) PRIMARY KEY, course_name VARCHAR(50)
);
```

```
CREATE TABLE Enrollments (
   enroll_id VARCHAR(20) PRIMARY KEY,student_id VARCHAR(15),course_id VARCHAR(15),
   FOREIGN KEY (student_id) REFERENCES Students(student_id),
   FOREIGN KEY (course_id) REFERENCES Courses(course_id)
);
```

--DEPARTMENTS TABLE

```
INSERT INTO Departments VALUES ('D01', 'Computer Science');
INSERT INTO Departments VALUES ('D02', 'Mechanical');
INSERT INTO Departments VALUES ('D03', 'Electrical');
```

--STUDENTS TABLE

```
INSERT INTO Students VALUES ('STU101A', 'Shubham Agarwal', 'D01');
INSERT INTO Students VALUES ('STU102B', 'Rahul Sharma', 'D02');
INSERT INTO Students VALUES ('STU103C', 'Amit Verma', 'D01');
INSERT INTO Students VALUES ('STU104D', 'Neha Kapoor', 'D03');
INSERT INTO Students VALUES ('STU105E', 'Priya Singh', 'D02');
INSERT INTO Students VALUES ('STU106F', 'Karan Mehta', 'D01');
```

--COURSES TABLE

```
INSERT INTO Courses VALUES ('CSE201', 'Database Management System');
INSERT INTO Courses VALUES ('CSE202', 'Operating System');
INSERT INTO Courses VALUES ('MTH101', 'Engineering Mathematics');
INSERT INTO Courses VALUES ('PHY110', 'Applied Physics');
```

--ENROLLMENTS TABLE

```
INSERT INTO Enrollments VALUES ('ENR001', 'STU101A', 'CSE201');
INSERT INTO Enrollments VALUES ('ENR002', 'STU101A', 'CSE202');
INSERT INTO Enrollments VALUES ('ENR003', 'STU101A', 'MTH101');
INSERT INTO Enrollments VALUES ('ENR004', 'STU102B', 'MTH101');
INSERT INTO Enrollments VALUES ('ENR005', 'STU103C', 'CSE201');
INSERT INTO Enrollments VALUES ('ENR006', 'STU103C', 'PHY110');
INSERT INTO Enrollments VALUES ('ENR007', 'STU106F', 'CSE201');
INSERT INTO Enrollments VALUES ('ENR008', 'STU106F', 'CSE202');
INSERT INTO Enrollments VALUES ('ENR009', 'STU106F', 'PHY110');
```

--VIEWING RECORDS OF DIFFERENT TABLES

```
SELECT * FROM DEPARTMENTS;
```

	dept_id [PK] character varying (10)	dept_name character varying (50)
1	D01	Computer Science
2	D02	Mechanical
3	D03	Electrical

SELECT * FROM STUDENTS;

	student_id [PK] character varying (15)	student_name character varying (50)	dept_id character varying (10)
1	STU101A	Shubham Agarwal	D01
2	STU102B	Rahul Sharma	D02
3	STU103C	Amit Verma	D01
4	STU104D	Neha Kapoor	D03
5	STU105E	Priya Singh	D02
6	STU106F	Karan Mehta	D01

SELECT * FROM ENROLLMENTS;

	enroll_id [PK] character varying (20)	student_id character varying (15)	course_id character varying (15)
1	ENR001	STU101A	CSE201
2	ENR002	STU101A	CSE202
3	ENR003	STU101A	MTH101
4	ENR004	STU102B	MTH101
5	ENR005	STU103C	CSE201
6	ENR006	STU103C	PHY110
7	ENR007	STU106F	CSE201
8	ENR008	STU106F	CSE202
9	ENR009	STU106F	PHY110

SELECT * FROM COURSES;

	course_id [PK] character varying (15)	course_name character varying (50)
1	CSE201	Database Management Syst...
2	CSE202	Operating System
3	MTH101	Engineering Mathematics
4	PHY110	Applied Physics

Step 1: Write queries to list students with their enrolled courses (INNER JOIN).

SELECT * FROM Students S INNER JOIN Enrollments E
ON S.student_id = E.student_id INNER JOIN Courses C
ON E.course_id = C.course_id;

	student_id character varying (15)	student_name character varying (50)	dept_id character varying (10)	enroll_id character varying (20)	student_id character varying (15)	course_id character varying (15)	course_id character varying (15)	course_name character varying (50)
1	STU101A	Shubham Agarwal	D01	ENR001	STU101A	CSE201	CSE201	Database Management Syst...
2	STU101A	Shubham Agarwal	D01	ENR002	STU101A	CSE202	CSE202	Operating System
3	STU101A	Shubham Agarwal	D01	ENR003	STU101A	MTH101	MTH101	Engineering Mathematics
4	STU102B	Rahul Sharma	D02	ENR004	STU102B	MTH101	MTH101	Engineering Mathematics
5	STU103C	Amit Verma	D01	ENR005	STU103C	CSE201	CSE201	Database Management Syst...
6	STU103C	Amit Verma	D01	ENR006	STU103C	PHY110	PHY110	Applied Physics
7	STU106F	Karan Mehta	D01	ENR007	STU106F	CSE201	CSE201	Database Management Syst...
8	STU106F	Karan Mehta	D01	ENR008	STU106F	CSE202	CSE202	Operating System
9	STU106F	Karan Mehta	D01	ENR009	STU106F	PHY110	PHY110	Applied Physics

STEP 2: Find students not enrolled in any course (LEFT JOIN).

```
SELECT s.student_id, s.student_name
FROM STUDENTS s
LEFT JOIN ENROLLMENTS e ON s.student_id = e.student_id
WHERE e.student_id IS NULL;
```

	student_id [PK] character varying (15)	student_name character varying (50)
1	STU104D	Neha Kapoor
2	STU105E	Priya Singh

STEP 3: Display all courses with or without enrolled students (RIGHT JOIN).

```
SELECT c.course_id, c.course_name, s.student_name
FROM STUDENTS s
RIGHT JOIN ENROLLMENTS e ON s.student_id = e.student_id
RIGHT JOIN COURSES c ON e.course_id = c.course_id;
```

	course_id character varying (15)	course_name character varying (50)	student_name character varying (50)
1	CSE201	Database Management Syst...	Shubham Agarwal
2	CSE202	Operating System	Shubham Agarwal
3	MTH101	Engineering Mathematics	Shubham Agarwal
4	MTH101	Engineering Mathematics	Rahul Sharma
5	CSE201	Database Management Syst...	Amit Verma
6	PHY110	Applied Physics	Amit Verma
7	CSE201	Database Management Syst...	Karan Mehta
8	CSE202	Operating System	Karan Mehta
9	PHY110	Applied Physics	Karan Mehta

STEP 4: Show students with department info using SELF JOIN or MULTIPLE JOINS.

```
SELECT s.student_id, s.student_name, d.dept_name
FROM STUDENTS s
INNER JOIN DEPARTMENTS d ON s.dept_id = d.dept_id;
```

	student_id character varying (15)	student_name character varying (50)	dept_name character varying (50)
1	STU101A	Shubham Agarwal	Computer Science
2	STU102B	Rahul Sharma	Mechanical
3	STU103C	Amit Verma	Computer Science
4	STU104D	Neha Kapoor	Electrical
5	STU105E	Priya Singh	Mechanical
6	STU106F	Karan Mehta	Computer Science

STEP 5: Display all possible student-course combinations (CROSS JOIN).

```
SELECT s.student_name, c.course_name
FROM STUDENTS sCROSS JOIN COURSES c;
```

	student_name character varying (50)	course_name character varying (50)
1	Shubham Agarwal	Database Management Syst...
2	Rahul Sharma	Database Management Syst...
3	Amit Verma	Database Management Syst...
4	Neha Kapoor	Database Management Syst...
5	Priya Singh	Database Management Syst...
6	Karan Mehta	Database Management Syst...
7	Shubham Agarwal	Operating System
8	Rahul Sharma	Operating System
9	Amit Verma	Operating System
10	Neha Kapoor	Operating System
11	Priya Singh	Operating System
12	Karan Mehta	Operating System
13	Shubham Agarwal	Engineering Mathematics
14	Rahul Sharma	Engineering Mathematics
15	Amit Verma	Engineering Mathematics
16	Neha Kapoor	Engineering Mathematics
17	Priya Singh	Engineering Mathematics
18	Karan Mehta	Engineering Mathematics
19	Shubham Agarwal	Applied Physics
20	Rahul Sharma	Applied Physics
21	Amit Verma	Applied Physics
22	Neha Kapoor	Applied Physics
23	Priya Singh	Applied Physics

I/O Analysis

Input:

The input consists of data stored in four relational tables:

- **Departments Table**

```
dept_id | dept_name
--------------------------
D01     | Computer Science
D02     | Mechanical
D03     | Electrical
```

- **Students Table**

```
student_id | student_name      | dept_id
----------------------------------------
STU101A    | Shubham Agarwal   | D01
STU102B    | Rahul Sharma      | D02
STU103C    | Amit Verma        | D01
STU104D    | Neha Kapoor       | D03
STU105E    | Priya Singh       | D02
STU106F    | Karan Mehta       | D01
```

- **Courses Table**

```
course_id | course_name
--------------------------------------
CSE201    | Database Management System
CSE202    | Operating System
MTH101    | Engineering Mathematics
PHY110    | Applied Physics
```

- **Enrollments Table**

```
enroll_id | student_id | course_id
----------------------------------
ENR001    | STU101A    | CSE201
ENR002    | STU101A    | CSE202
ENR003    | STU101A    | MTH101
ENR004    | STU102B    | MTH101
ENR005    | STU103C    | CSE201
ENR006    | STU103C    | PHY110
ENR007    | STU106F    | CSE201
ENR008    | STU106F    | CSE202
ENR009    | STU106F    | PHY110
```

Output:

- **Step 1 – Inner Join Operation:**
  Displayed only those students who are enrolled in courses, ensuring matching records from all related tables.

- **Step 2 – Left Join Operation:**
  Identified students who are not enrolled in any course by showing unmatched records from the Students table.

- **Step 3 – Right Join Operation:**
  Displayed all courses along with enrolled students, ensuring that even courses without students would be included.

- **Step 4 – Multiple Join Operation:**
  Combined Students and Departments tables to provide complete student details along with their department information.

- **Step 5 – Cross Join Operation:**
  Generated all possible combinations of students and courses, demonstrating the concept of Cartesian product.

Learning Outcomes

- **Understanding of JOIN Operations:**
  Gained knowledge of different types of SQL joins such as INNER JOIN, LEFT JOIN, RIGHT JOIN, and CROSS JOIN and their practical applications.

- **Data Integration Skills:**
  Learned how to combine data from multiple related tables (Students, Courses, Enrollments, Departments) to extract meaningful information.

- **Handling Matched and Unmatched Records:**
  Developed the ability to identify both matching and non-matching records using INNER and OUTER joins.

- **Query Writing and Optimization:**
  Improved skills in writing efficient SQL queries to retrieve structured and relevant data from relational databases.